

Technical Documentation

System architecture, explainable matching, readiness scoring, evaluation and multi-campus deployment

Document context	Scope
Prepared	9 October 2026 CampusLink repository review
Implementation	Next.js frontend, Express API, SQLite / PostgreSQL persistence
Evaluation evidence	Fresh isolated synthetic placement demonstration; saved synthetic model artifact
Audience	Project reviewers, developers, placement teams and deployment operators

How to read this report

This report explains the current implementation and its observed demonstration results. Future evaluation and scaling work is explicitly identified as proposed. Scores measure recorded preparation or role fit; they are not guarantees of hiring. No public dataset was downloaded or evaluated for this report.

The three central capabilities are student readiness profiling, recruiter-student matching with conflict-aware scheduling, and placement monitoring through offers, document verification and analytics.

Evidence snapshot: a successful **npm run demo:placement** run on 9 October 2026. It uses synthetic records in an isolated in-memory database and sends no external email.

01 / Overview and contents

CampusLink coordinates placement activity among students, recruiters, institutional placement teams and administrators. Its main purpose is to turn profiles, recruiter requirements and preparation evidence into an explainable workflow, then preserve the operational record from applications to joining.

Section	Coverage
02-03	System architecture and an end-to-end placement request
04-05	Data model, persistence, security and campus isolation
06-08	Readiness, eligibility and fit scoring with worked examples
09-10	Local NLP, TF-IDF matching and optional prediction/coaching models
11	Scheduling, offers, notifications and operational analytics
12-14	Synthetic dataset, measured accuracy and performance interpretation
15	Proposed broader evaluation and public-dataset suitability
16-17	Multi-campus scalability and deployment procedure
18-19	Operations, limitations, improvement priorities and references

Implementation status

Implemented: role dashboards, persistent authentication, campus-scoped workflows, rule-based readiness and eligibility, weighted role fit, TF-IDF relevance, drive lifecycle management, conflict detection, offers, private document records, notifications and analytics.

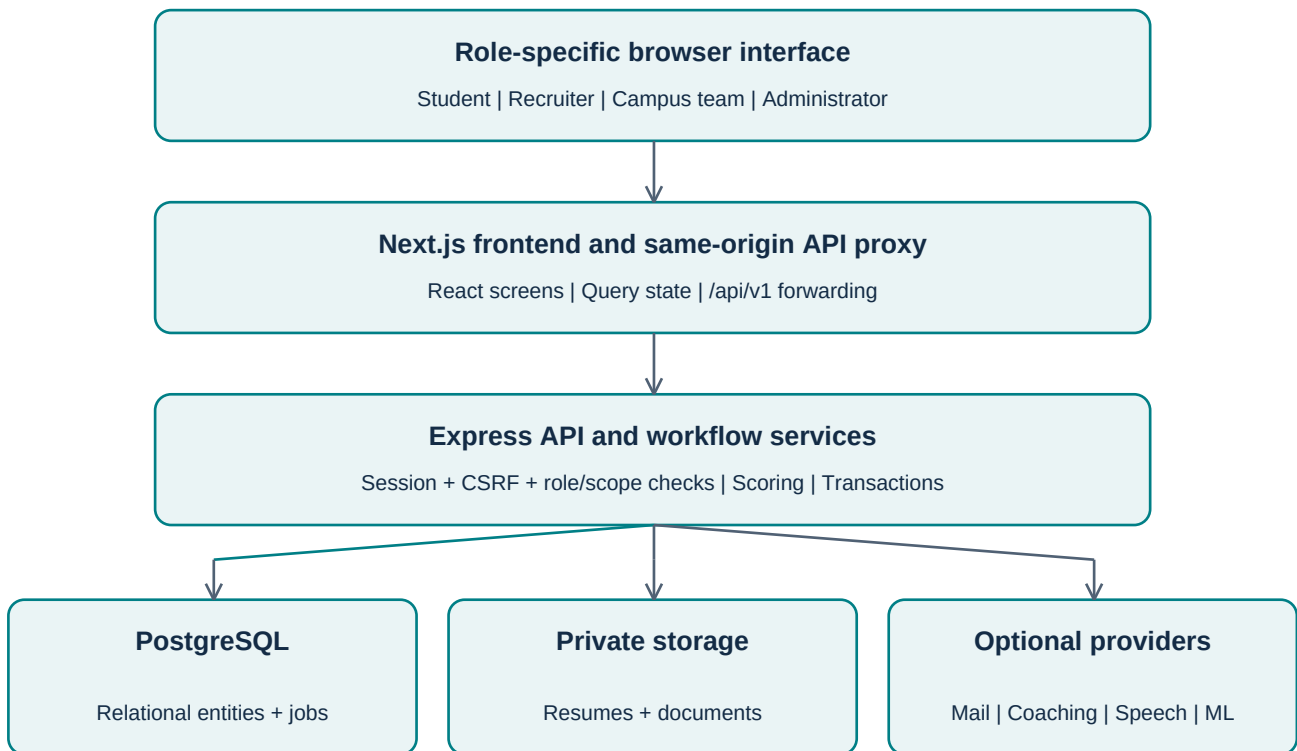
Optional: external generative coaching, voice transcription and remote ML analysis. Configuration and consent determine whether these services are called. Local preparation analysis remains available when optional providers are absent or fail.

Not established by current evidence: validated real-world placement prediction, accuracy on public hiring datasets, production concurrency limits, p95 API latency or multi-region disaster recovery performance.

Three meanings of a score

Readiness describes the student across preparation categories. Fit describes one student against one drive. Outcome probability, when a qualifying model exists, estimates association with historical placement labels. These outputs answer different questions and must remain separately labelled.

02 / System architecture



The browser uses role-specific Next.js screens. Requests to the frontend origin under /api/v1 are forwarded to the Express backend. This proxy arrangement lets browser sessions remain associated with the frontend domain while the backend runs as a separate service.

Express authenticates requests, validates origin and CSRF requirements, checks roles and ownership, and invokes workflow services. Business rules and database transactions are authoritative on the server. Interface controls improve usability but do not replace server authorization.

PostgreSQL is the production persistence layer; SQLite is the development fallback. Private storage keeps uploaded bytes outside the public website. Stored mail and cleanup jobs support asynchronous work. Optional AI, speech and ML providers sit outside the core placement transaction path.

Why this structure fits the project

A modular application keeps profile, matching, scheduling and offer rules together while deployment separates frontend rendering, API execution and storage. It avoids introducing independent distributed services before the workflow has enough traffic to justify them.

Implementation evidence: README.md; server/app.ts; server/db.ts; server/services.ts; server/storage.ts; vercel.json; render.yaml.

03 / Request and workflow lifecycle

Stage	What happens and why
1. Profile	Student records academics, skills and projects. Assessment attempts provide additional evidence. Resume extraction proposes skills for review; it does not automatically verify them.
2. Partnership / drive	Recruiter requests campus access and submits structured drive criteria. Campus approval establishes a permitted recruiting relationship.
3. Eligibility	Campus, course, branch, graduation year, CGPA, backlogs and optional mandatory skills are checked. Free-text conditions require recorded approval.
4. Matching	Eligible students receive weighted fit and lexical relevance scores. Recruiters receive rankings, explanations and skill gaps.
5. Schedule / selection	Campus proposes schedules, recruiter confirms and campus finalizes. Interviews, results and application state updates follow guarded workflows.
6. Offer / joining	Recruiter releases an offer after selection. Student responds; campus verifies documents before recording joining.
7. Monitoring	Stored notifications, queued emails, reminders, audit events and scoped analytics expose current progress.

Example: a student applies for a drive

The browser sends an authenticated service request. The server identifies the account from its session, checks that the account and campus are permitted, loads the current drive, and recomputes eligibility. A stale browser score cannot bypass the checks. The application and associated notification records are committed through the persistence layer.

Selection and post-selection actions have distinct authority. A recruiter records results and releases the offer; the student controls acceptance or deferral; campus staff control document verification and joining. This separation preserves accountability.

Implementation evidence: `server/services.ts`; `server/recruitment.ts`; `server/workspace.ts`; `server/API.md`; `src/services/drive.domain.ts`.

04 / Data model and persistence

Entity group	Relationships and integrity
Identity / scope	Accounts have stable IDs, role and campus/owner relationships. Sessions and password-reset tokens reference accounts.
Student evidence	Profiles, skills, projects, assessment attempts, documents and preparation history reference the student account.
Recruiting	Drives reference campus and recruiter. Rounds and schedules reference drives. Applications connect student and drive.
Selection	Results reference application and round. Interview slots reference drive, round, recruiter and optional student.
Outcomes	Offers reference student and, where available, application and recruiter. Document records remain scoped to their student.
Operations	Notifications reference recipients. Audit records preserve actors and details. Mail jobs, cleanup jobs, locks and migration markers support operations.

Core entities are stored as relational rows with foreign keys, unique constraints and campus/owner columns. Flexible attributes remain in JSON-validated payloads, while important identifiers, statuses, scores and dates also have SQL columns. The adapter writes query columns and payloads together.

One application per student/drive, one result per application/round and one schedule per drive prevent common duplicates. Composite foreign keys protect student, application, drive and campus relationships, including preventing a resume reference from pointing to another student's document.

Consistency and migration

Aggregate responses are reconstructed from canonical rows rather than duplicated independent application lists. Related data is loaded in batches. PostgreSQL aggregate reads use repeatable-read snapshots; write transactions acquire the shared placement lock. SQLite serializes transactions in the local process.

The relational cutover preserves a legacy snapshot and adds guards against old writers. It is a one-way cutover, so old backend instances must be stopped first. A failed conversion rolls back instead of fabricating missing records. Backups and migration verification remain essential.

Implementation evidence: `server/schema.ts`; `server/relational-store.ts`; `server/db.ts`; `server/DATABASE.md`.

05 / Security, permissions and isolation

Authentication and request protection

Authentication uses random HTTP-only session cookies with hashed persistent session records. Production cookies are secure. Authenticated writes require the session CSRF token, and origin checks constrain which frontend may send requests. Browser-supplied identity or role values do not establish authority.

Campus and recruiter boundaries

Campus teams operate on students registered to their campus. Recruiters operate on owned drives and authorized applicant pools. Student actions remain tied to the authenticated student. Where a team supplies X-Student-ID, the server checks the target against the actor's permitted scope before performing the operation.

There are two related controls: API authorization uses campus IDs and ownership; the eligibility utility also compares the student's campus name with the drive campus. The name check alone should never serve as tenant authorization. Immutable campus IDs and relational constraints are the stronger boundary.

Documents and evidence integrity

Private file access passes through authorized download endpoints. Uploads have a 10 MB limit and file-signature checks for supported PDF and image formats. Stored documents are not served from public/. Profile patches cannot directly assign verified skills, XP, identity or campus membership. Assessment answers are graded on the server.

External processing and secrets

Server credentials stay in backend environment settings. External analysis requires student consent; generative consent is tied to the selected provider. ML text redaction removes email and telephone patterns, but is not complete anonymization. Store and transmit only the minimum fields needed for the task.

PostgreSQL tables use a server-only RLS posture: direct client access has no policies, and the trusted backend role accesses data. This does not automatically implement database-level per-campus policies for the backend; API authorization is still critical.

Implementation evidence: `server/auth.ts`; `server/app.ts`; `server/services.ts`; `server/ml-client.ts`; `server/ai-provider.ts`; `server/API.md`; `server/DATABASE.md`.

06 / Readiness scoring algorithm

Readiness is a deterministic weighted index from 0 to 100. It summarizes recorded preparation evidence; no fitted machine-learning weights are used in this calculation. Each category is converted to a score and then multiplied by its configured weight.

Category	Computation on the 0-100 scale	Weight
Verified skills	Rounded percentage of recorded skills marked verified; zero if no skills exist.	30%
Academics	Clamp(CGPA x 10 - active backlogs x 10, 0, 100).	20%
Projects	min(100, number of recorded projects x 35).	15%
Aptitude	Rounded average of attempts with type containing aptitude.	15%
Communication	Rounded average of attempts with type containing communication or soft.	10%
Interview	Rounded average of attempts with type containing interview.	10%

$$R = \text{round}(0.30 V + 0.20 A + 0.15 P + 0.15 T + 0.10 C + 0.10 I)$$

The output includes category scores, weights and a recorded flag. Missing attempts contribute zero and the weights are not redistributed. Factors below 60 are highlighted for preparation support. A recorded zero and missing evidence can both have zero contribution, so the recorded flag is necessary to interpret them correctly.

Score interval	Current label
0-44	Not Ready
45-69	Developing
70-84	Ready
85-100	Highly Employable

A student with all skills verified, CGPA 9, no backlogs, three projects and 85 in each practice category has $R = \text{round}(30 + 18 + 15 + 12.75 + 8.5 + 8.5) = 93$. This is the demonstration's strong-profile result.

Implementation evidence: src/utis/scoring.ts: readiness; output/placement-demo-report.json.

07 / Eligibility gates and skill gaps

Eligibility is evaluated before a meaningful fit score is produced. All applicable checks must pass. A failure produces explanations and a zero role-fit/hybrid score, regardless of preparation evidence.

Gate	Implemented rule
Campus	Student and selected drive campus names must agree after trimming and lowercasing; server ID-based scope is separately enforced.
Course / branch	Course must be in the allowed list. Branch labels are normalized before checking the allowed branches.
Graduation year	Recorded student year must appear in the drive's allowed graduation-year list.
CGPA / backlogs	CGPA must be at least the minimum; active backlogs must not exceed the allowance, which defaults to zero.
Mandatory skills	When requireSkills is enabled, every required skill must be recorded after normalization. Verification is not required for this gate.
Additional conditions	Nonempty extra eligibility text requires the student's ID in the recorded eligibility approvals.

Recorded skills versus verified skills

A recorded unverified skill can satisfy the mandatory-skills gate but receives partial credit in required-skill fit. This allows a developing candidate to remain eligible while clearly showing that evidence is weaker. When mandatory skills are disabled, missing skills affect fit and preparation advice rather than creating an automatic skill-based rejection.

Gap generation

Every required skill without verification becomes a gap. If it is recorded, the suggested action is an assessment to verify it. If absent, the suggested action is learning fundamentals and adding a project. These instructions describe a route to stronger evidence, not a promise that taking the action will secure an offer.

Boundary behavior to evaluate

Test exact CGPA and backlog thresholds, alternative branch names, empty criteria, stale approvals and cross-campus requests. Extraction from job text remains a reviewed suggestion: structured saved requirements are what eligibility actually uses.

Implementation evidence: `src/utls/placement.ts: checkEligibility`; `src/utls/skills.ts`; `src/utls/scoring.ts: fit`.

08 / Role fit and hybrid matching

Fit factor	Implemented calculation	Weight
Required skills	Average credits: verified = 1, recorded unverified = 0.5, absent = 0; converted to percentage. No required skills gives 100.	35%
Preferred skills	Percentage recorded, regardless of verification. If no preferred skills are specified, use required alignment.	10%
Assessments	Rounded average of all supplied assessment history scores; absent history gives zero.	20%
Project relevance	Projects whose title/description includes any required skill text; $\min(100, \text{count} \times 50)$.	15%
Certifications	$\min(100, \text{recorded certification count} \times 50)$.	5%
Academics	$\min(100, \text{CGPA} \times 10)$; backlog effects are handled by eligibility.	15%

```

F = round(sum of fit factor score x weight), if eligible
H = round(0.85 F + 0.15 x 100 x cosine_similarity)
F = H = 0, if eligibility fails

```

Candidates are sorted by H in descending order. The output also retains readiness, rule-factor explanations, eligibility reasons, lexical similarity and skill gaps. The implementation's label comes from rule fit F; it is not recomputed from H. Displaying these as separate fields prevents apparent inconsistency.

Worked example from the synthetic run

The strong frontend candidate has required alignment 100, preferred alignment 100, assessments 85, project relevance 50, certifications 0 and academics 90. Its rule fit is $\text{round}(35 + 10 + 17 + 7.5 + 0 + 13.5) = 83$. Its final hybrid score is **74** after the lexical component; readiness remains **93**. Those differences follow from the three formulas, not an error.

Weights are fixed design choices rather than empirically optimized hiring predictors. Counts of projects/certificates and averages of unrelated attempts can be gamed or misrepresent depth. Future validation should test role-specific assessment evidence and project quality.

Implementation evidence: `src/utls/scoring.ts`: fit; `server/services.ts`: rankedCandidates; `output/placement-demo-report.json`.

09 / NLP and TF-IDF relevance

Local skill and requirement extraction

A reviewed dictionary maps aliases such as `reactjs` to `React` and `postgres` to `SQL`. Boundary-aware regular expressions reduce accidental matches inside longer words. Requirement extraction recognizes CGPA patterns, a limited set of branch abbreviations, graduation years in the 2020-2049 range, and phrases meaning no or zero backlogs. Results require review.

Resume analysis identifies dictionary skills and checks for measurable outcomes, professional links, project/experience sections and sufficient text. It returns suggestions and word count. Extraction alone never adds or verifies a student skill.

How lexical matching works

The query combines required skills with the drive description. Candidate documents concatenate skill names, project titles and project descriptions. Tokenization lowercases text, normalizes known aliases, removes a small stop-word list and extracts letter-led tokens of at least two characters. It does not perform learned embedding-based semantic understanding.

```
corpus = [query, candidate_1, ..., candidate_N]
TF(t,d) = count(t,d) / max(1, token_count(d))
IDF(t) = ln((corpus_size + 1)/(document_frequency(t) + 1)) + 1
v_d(t) = TF(t,d) x IDF(t)
sim(q,d) = dot(v_q,v_d) / (norm(v_q) x norm(v_d))
```

The implementation uses smoothed inverse document frequency and length-normalized frequency. A zero denominator returns zero similarity. Multiplication by 100 puts cosine relevance on the same numerical scale as rule fit before weighting. General vector-space theory is described in the Stanford information retrieval text [E1].

Interpretation and limitations

Lexical matching is transparent and inexpensive for small pools, but may miss paraphrases and overreward copied keywords. IDF is recomputed using the current candidate pool and query, so a candidate score can change when that pool changes. Score reproducibility requires recording the pool, criteria, dictionary and scoring version.

Implementation evidence: `server/nlp.ts`; `server/services.ts`: `rankedCandidates`; `server/API.md`.

10 / Prediction and optional AI models

Local historical-outcome logistic regression

The optional trainer consumes six normalized readiness features and a binary placed label. It adds an intercept and learns seven weights. Rows need at least 60 labelled outcomes across three cohorts; both training and held-out data must contain placed and unplaced students. Cohort strings are sorted, with the last cohort held out. Cohort names must therefore sort chronologically.

```
x = [1, V/100, A/100, P/100, T/100, C/100, I/100]
p(placed | x) = sigmoid(w dot x)
sigmoid(z) = 1 / (1 + exp(-clamp(z, -30, 30)))
w_i <- w_i - 0.15 x (mean_gradient_i + 0.01 w_i)
(intercept has no regularization; 1,800 full-batch epochs)
```

The update corresponds to logistic-loss training with L2 regularization on feature weights. Evaluation uses a 0.5 threshold and reports accuracy, precision, recall and mean Brier score. Feature contributions describe associations in the fitted model, not causal effects. There is no implemented hyperparameter search or calibration certification.

What the saved artifact actually proves

The supplied placement.json is marked **synthetic**, with 600 training rows and 300 test rows, training cohorts 2023/2024 and test cohort 2025. Stored metrics are 69.0% accuracy, 69.89% precision, 50.0% recall and 0.20899 Brier score. These numbers are artifact metadata, not a training experiment rerun for this document. The original row-level dataset was not verified here.

The local insight function rejects non-historical provenance. The remote placement connector also rejects non-historical provenance. A provenance flag alone is not proof of independent validation: operational release should additionally require audited datasets, model integrity checks and approved evaluation evidence.

Coaching, speech and external ML

Gemini/OpenAI structured generation supplies preparation advice when configured and consented. Voice transcription has a separate provider configuration. A remote ML connector validates response schemas, enforces bounded requests and falls back to local guidance on failure. These integrations do not establish that a production predictive service is deployed.

Implementation evidence: server/ml.ts; server/models/placement.json; server/ml-client.ts; server/ai-provider.ts; server/speech.ts.

11 / Scheduling and placement operations

Drive state management

The main path proceeds through Draft, Submitted, Under Review, Scheduling, Awaiting Recruiter Confirmation, Confirmed, Active, In Progress and Completed. Campus finalization occurs before activation. Changes requested, rejection, cancellation and application closure provide guarded alternative transitions. Actor and state checks prevent invalid actions.

Conflict-aware scheduling

A schedule has reporting, talk, assessment, interview and end times plus venue, lab, rooms and system capacity. Validation checks ordering and capacity. Conflict checks consider overlapping campus resources, recruiter availability and affected student cohorts. Interview scheduling also checks candidate, room and panel collisions. Suggestions are rechecked when saved because another reservation may have appeared.

```
Two intervals overlap when:  
start_A < end_B AND start_B < end_A  
Then test whether an exclusive resource or participant is shared.
```

This interval rule explains the general overlap principle. The concrete implementation also applies drive-state, scope and schedule-field rules. It is a conflict-aware workflow, not a global optimization algorithm that guarantees the best arrangement of all drives.

Offer and document lifecycle

Results advance qualified candidates to the offer stage. Recruiters create Full-time, PPO or Internship-conversion offers. Students may accept, decline, defer or withdraw within permitted rules. Campus staff record joining only after a verified resume and no outstanding unverified documents. The demonstration exercises deferral, acceptance, blocked premature joining and successful joining after verification.

Notifications and analytics

Workflow changes store notifications and targeted email jobs transactionally. Reminders use daily deduplication; cleanup is queued after document replacement. Analytics summarize readiness, applications, offers, acceptance, joining, document status and package figures within authorized scope. These operational counts are distinct from prediction quality metrics.

Implementation evidence: `src/services/drive.validation.ts`; `src/services/drive.domain.ts`; `server/recruitment.ts`; `server/reminders.ts`; `server/services.ts`.

12 / Synthetic evaluation dataset

The reproducible placement demonstration creates 13 artificial students in two colleges: 12 in the main campus and one in another campus. It uses an isolated in-memory SQLite database. Resume records are simulated metadata, not actual uploaded or parsed PDFs. No email delivery worker is started.

Profile group	Purpose in the demonstration
3 strong specialists	Frontend: React/TypeScript; backend: Java/SQL; data: Python/SQL. Verified skills, three projects and three scores of 85.
3 developing specialists	Same role-specific skills recorded but unverified; no project or assessment evidence.
1 all-rounder	All five tested skills recorded and unverified; checks the value of evidence relative to breadth.
1 missing-skills student	No skills; checks the mandatory-skills gate.
5 edge profiles	CGPA 5, Mechanical branch, graduation year 2026, two active backlogs and another campus.

Default synthetic academic values are MCA, CSE, graduation year 2027 and CGPA 9 with no active backlogs. Three simulated drives require MCA/CSE/2027, CGPA at least 7, zero allowed backlogs and their mandatory skill pair. Each drive has three manually specified eligible candidates and one expected best candidate.

Drive	Required skills	Expected leader
Frontend Developer	React, TypeScript	web-strong
Backend Developer	Java, SQL	backend-strong
Data Analyst	Python, SQL	data-strong

Labels and leakage risk

Eligibility labels are declared separately from checkEligibility in the fixture. They still follow the same small set of designed rules, so the experiment primarily measures rule conformance. The deliberately strong specialists make ranking comparatively easy. There are no independent recruiter relevance judgements or real employment outcomes.

Implementation evidence: scripts/placement-demo.ts; output/placement-demo-report.json.

13 / Measured accuracy and scoring results

Metric	Observed result	Meaning
Eligibility accuracy	39 / 39 = 100%	Agreement with all manually labelled student-drive pairs.
Confusion matrix	TP 9, TN 30, FP 0, FN 0	Derived from three positive labels per drive and 39 total pairs.
Eligibility precision / recall	100% / 100%	On this fixture only; all eligible cases are accepted by the rules.
Top-one ranking accuracy	3 / 3 = 100%	Expected strong specialist leads each drive.
Strong readiness	93 / 100	Deterministic readiness calculation, not outcome probability.
Hybrid ranking by drive	74, 40, 34	Strong specialist, developing specialist, all-rounder in each of the three drives.

What 100% means here

The rules passed the selected edge cases and the ranking favors recorded evidence in these examples. The denominator is small: only three ranking tasks and correlated pairs that reuse the same students. It would be misleading to describe this as 100% placement prediction accuracy.

For scale awareness, a conventional Wilson 95% interval for 39/39 is approximately 91.0%-100%; for 3/3 it is approximately 43.9%-100%. These illustrative intervals assume independent trials, which this designed, reused fixture does not provide. They do not estimate real-world generalization.

Scoring evaluation beyond agreement

The strong-profile readiness of 93 and rule-fit example of 83 are consistent with the documented formulas. The demonstration exposes factor explanations and skill gaps, but does not contain independent target readiness scores, expert-rated continuous fit labels or ground-truth NLP relevance. Consequently, readiness MAE, score correlation, NLP precision and NDCG cannot be claimed from this run.

Additional workflow assertions passed

The run also checks drive and interview conflicts, campus isolation, blocked ineligible application, explanation availability, document verification before joining, offer deferral and repeated-reminder deduplication. These are positive scenario checks, not exhaustive security or concurrency coverage.

Implementation evidence: Fresh npm run demo:placement; scripts/placement-demo.ts; output/placement-demo-report.json.

14 / Performance evaluation and limits

The fresh successful demonstration recorded **307.61 ms** for its timed section. This measurement uses the script's `performance.now` timer, not browser response time or external HTTP latency.

Exact timing boundary

Timing starts after schema migration, campus/account creation and student fixture seeding. It includes the three-drive loop: recruiter-campus approval, drive creation and activation, schedule proposals and checks, all 39 eligibility comparisons, eligible interests/applications, database operations and three ranking computations. It stops before later interview-result, offer, reminder, document-verification and analytics checks.

Performance item	Evidence status
Timed demonstration loop	307.61 ms in one local run with in-memory SQLite.
End-to-end HTTP latency	Not measured; no network request timing in this fixture.
Production PostgreSQL throughput	Not measured; SQLite timing cannot substitute for hosted database latency.
p50 / p95 / p99	Not measured; one timing sample is insufficient.
Concurrent scheduling safety	Sequential assertions pass; race and contention load testing remains required.
External provider performance	Not measured; no coaching, speech or external ML benchmark was performed.

Current computational costs

Rule scoring grows with candidate count and the amount of skills, projects and history inspected. TF-IDF builds a vocabulary for the whole pool and repeatedly scans token lists for each vocabulary term. The current approach can approach $O(N \times V \times L)$ work, where N is document count, V vocabulary size and L average document length. Dense term maps also use roughly $O(N \times V)$ memory. Sorting costs $O(N \log N)$.

Workspace lookup currently searches the workspace list for each profile, creating potential quadratic overhead. Frequent dashboard reads and a shared placement write lock can become database bottlenecks before frontend rendering does. These are code-based scaling inferences, not measured throughput limits.

Implementation evidence: `scripts/placement-demo.ts` timing boundaries; `server/nlp.ts`; `server/services.ts`; `server/db.ts`.

15 / Broader evaluation plan

Proposed work; not executed: extend the fixture with seeded, reproducible variation across campuses, roles, skills, CGPA boundaries, missing fields and project vocabulary. Preserve expected eligibility labels independently and publish failed-case examples as well as aggregate metrics.

Match relevance and readiness quality

Obtain blinded recruiter ratings for eligible candidate-drive pairs, such as 0 = unsuitable through 3 = highly relevant. Compare rule fit, TF-IDF alone and the 85/15 hybrid on identical pools. Report Precision@k, Recall@k and NDCG@k per drive and campus, with uncertainty. Readiness quality needs separately defined expert rubrics; use MAE and rank correlation only when meaningful reference scores exist.

```
Precision@k = relevant candidates in top k / k
Recall@k = relevant candidates in top k / all relevant candidates
DCG@k = sum_i((2^relevance_i - 1)/log2(i + 1))
NDCG@k = DCG@k / ideal_DCG@k
Brier = mean((predicted_probability - outcome)^2)
```

Historical outcome evaluation

Collect real, consented cohort outcomes with feature timestamps preceding the placement outcome. Deduplicate students across folds, keep the latest cohort held out and add campus holdouts to test transfer. Compare with majority and simple logistic baselines. Report accuracy, precision, recall, F1, PR-AUC, ROC-AUC, Brier score and calibration. Define whether placed means offer, acceptance or joining and retain the follow-up horizon.

Public dataset suitability

UCI Student Performance [E2] is a public education dataset about secondary-school grades. It is useful as an example of academic prediction data, but does not supply CampusLink's job requirements, verified skills, recruiter relevance labels or campus placement outcomes. It cannot honestly validate this matching system without a separate labelled mapping task. No public-dataset accuracy is reported here.

Load and robustness study

Use synthetic cohorts at 1,000, 10,000 and 100,000 students as proposed test sizes, not capacity promises. Exercise realistic reads/writes, concurrent schedule claims, malformed uploads and provider timeouts. Record hardware, database size, replica count, p95 latency, errors, memory, connection wait and lock wait. Repeat runs and retain raw measurements.

Implementation evidence: Evaluation recommendations derived from current implementation; external sources [E1]-[E2].

16 / Multi-campus scalability approach

Current multi-campus foundation

One authoritative API and shared PostgreSQL database preserve cross-campus scheduling and recruiter ownership rules. Campus and owner columns support scoped access and indexed reads. Frontends across institutions can use the same API. Recruiter availability checks need a shared view because a recruiter may participate in several campuses.

Stage 1: strengthen one deployment

Use PostgreSQL, private object storage and a persistent backend process. Add pagination, drive/campus filters, direct SQL aggregation and bounded responses for large cohorts. Replace per-profile workspace searches with an ID map. Precompute token frequencies and sparse vectors; cache matching only with keys that include drive revision, candidate evidence revision and pool/version information.

Stage 2: add API replicas safely

Sessions and entity state are already persistent, but process-local rate limits and worker behavior need a shared coordination design. Use a shared rate-limit store and durable job claims with leases/idempotency before concurrent workers run. Externalize workers so API replicas can scale without duplicating delivery. Monitor database connection count: the current pool limit is four connections per API process.

Stage 3: reduce measured bottlenecks

The shared placement lock serializes writes across API instances. Extra replicas may improve reads but do not remove this write constraint. A later design could replace it with transaction-safe resource locks or database reservation constraints, acquired in a deterministic order. Cross-campus recruiter collisions must remain protected. Make this change only after race tests demonstrate equivalent correctness.

Scale concern	Proposed control
Tenant isolation	ID-scoped queries, composite constraints, authorization tests and per-campus metrics.
Fair resource usage	Per-campus quotas and bounded matching jobs to prevent one large campus dominating.
Large analytics	Incremental aggregates or reporting replicas when measured read traffic justifies them.
Reliability	Managed backups, restore drills, provider timeouts, queue retry visibility and failure alerts.

Campus partitioning or separate databases may eventually suit institutional requirements, but introduce cross-campus recruiter coordination and reporting complexity. No maximum supported campus count is established by the supplied demonstration.

Implementation evidence: `server/db.ts`; `server/relational-store.ts`; `README.md`; `server/DATABASE.md`. Scaling stages are proposed.

17 / Deployment procedure

The repository deployment approach uses Vercel for the frontend and Render for the Express backend, with PostgreSQL, private Supabase storage and Resend delivery. This describes the checked-in configuration; it is not evidence that a particular live environment has been deployed or tested.

Component	Required configuration
Frontend	NEXT_PUBLIC_APP_ENV=api; NEXT_PUBLIC_API_URL=/api/v1; API_INTERNAL_URL set to the backend HTTPS origin, without /api/v1.
Backend	NODE_ENV=production; FRONTEND_URL set to the exact frontend HTTPS origin; hosting-provided PORT; listen on 0.0.0.0.
Database	DATABASE_URL and provider-compatible CA configuration. Validate TLS certificate and hostname.
Private storage	STORAGE_PROVIDER=supabase; SUPABASE_URL; SUPABASE_SECRET_KEY; private bucket, normally campuslink-private.
Email	EMAIL_PROVIDER=resend; RESEND_API_KEY; verified EMAIL_FROM.
Optional providers	AI, speech and ML settings belong only on the backend; student consent is separately required.

Build, migrate and release

1. Back up the production database and confirm credentials, exact origins and bucket privacy. If migrating legacy storage, stop old backend instances before cutover. Preserve existing environment secrets.
2. Install locked dependencies and run frontend/server type checks, lint and builds. Build the backend with npm run server:build. Run migration and database verification against the intended environment. The isolated local demonstration does not verify a hosted PostgreSQL migration.
3. Start the backend with npm run server:start and confirm /api/v1/health. Build/deploy the frontend with npm run build. API_INTERNAL_URL affects the built proxy, so a backend-origin change requires a frontend redeployment.
4. Smoke-test sign-in, approvals, cross-campus rejection, private upload/download, eligible and ineligible applications, schedule conflicts, offer response, document verification and email retry behavior. Check observability before opening access to institutions.

The checked-in Render manifest selects a free plan. Evaluate an always-on service for operational deployment: suspended processes delay requests and stop in-process reminder/mail work. Hosting-plan capacity and provider settings need verification at release time.

Implementation evidence: README.md; render.yaml; vercel.json; server/config.ts; server/DATABASE.md.

18 / Operations and improvement priorities

Monitoring and recovery

Track request latency and failure rates, database connection wait, transaction duration, placement-lock wait, matching duration, queue age, mail failures and storage-cleanup errors. Include correlation IDs and actor/campus metadata without logging private transcripts, session tokens or uploaded document contents. Readiness and placement conversion should be operational metrics, not model-accuracy substitutes.

Back up the database and private storage, then rehearse restoring them together so document records still resolve to file objects. Proposed recovery objectives must be selected with institutions and verified by timed restore exercises. The retained legacy records table is only a cutover snapshot and cannot recover later activity.

Release gates

Require passing schema/migration checks, scoped authorization tests, scheduling race tests, provider-failure handling and private-storage smoke tests. For outcome prediction, also require verified historical provenance, leakage checks, subgroup evaluation and calibration review. Keep a previous application build and database recovery path, with compatibility reviewed before rollback.

Priority	Action and expected benefit
P1: evidence quality	Define target labels and collect real historical/recruiter-rated evidence before accuracy claims or predictive release.
P1: concurrency	Test simultaneous bookings and job retries; implement shared claims before enabling multiple workers.
P2: scoring quality	Review role-specific assessments, project quality, certificate relevance and evidence freshness.
P2: computational efficiency	Filter/paginate early, map workspaces by ID and use sparse precomputed lexical features.
P2: explainability	Show readiness, rule fit and hybrid rank separately, with criteria/version and missing-evidence indicators.
P3: advanced models	Compare improved lexical retrieval or learned models only after an independent benchmark exists.

Known limits

Current weights and labels are heuristic. The local skill dictionary is limited, project relevance is keyword-based and certification counts do not prove quality. Historical insight relies on model-file contents and provenance flags rather than a full audited model registry. Scaling recommendations are design proposals; this report does not claim production capacity or public-data hiring accuracy.

19 / Reproducibility and references

Repeat the observed demonstration

```
npm run demo:placement
# Generated result: output/placement-demo-report.json
# Fixtures and assertions: scripts/placement-demo.ts
```

Run from the repository root using the project's configured Node.js 24 environment and installed dependencies. The fixture uses in-memory storage and avoids application-account changes. Timing varies by machine and run. The command overwrites its generated report; retain a dated copy when comparing experiments.

Local implementation reference map

Source	What it supports
README.md; package.json	Project structure, capabilities, configuration, scripts and deployment intent.
src/utlils/scoring.ts; placement.ts; skills.ts	Readiness weights, fit factors, thresholds, normalization and eligibility.
server/nlp.ts; services.ts	Dictionary extraction, tokenization, cosine similarity and hybrid ranking.
server/ml.ts; models/placement.json; ml-client.ts	Logistic training, saved artifact metadata and prediction provenance checks.
server/db.ts; schema.ts; relational-store.ts; DATABASE.md	Transactions, migrations, canonical rows, constraints and batching.
server/auth.ts; app.ts; API.md; ai-provider.ts	Sessions, permissions, service contracts, privacy and optional providers.
scripts/placement-demo.ts; output/placement-demo-report.json	Synthetic profiles, expectations, timing boundary and observed results.
drive.validation.ts; recruitment.ts; reminders.ts; render.yaml; vercel.json	Scheduling, post-selection workflows and checked-in hosting configuration.

External primary references

[E1] Manning, Raghavan and Schütze, *Introduction to Information Retrieval*. TF-IDF and vector-space cosine similarity. [Stanford information retrieval textbook](#), Chapter 6. The exact CampusLink formula is verified from code rather than assumed from the textbook.

[E2] UCI Machine Learning Repository, *Student Performance*, dataset 320. [Official dataset record](#). Referenced only to explain dataset suitability; no public-dataset experiment was performed. External references checked on 9 October 2026.

Evidence labels throughout this document separate implementation facts, freshly observed fixture results, stored artifact metadata and proposed future work. This distinction is necessary for reviewable technical claims.